



This chapter covers

- The inspiration for evolutionary algorithms
- Solving problems with evolutionary algorithms
- Understanding the life cycle of a genetic algorithm
- Designing and developing a genetic algorithm to solve optimization problems

What is evolution?

When we look at the world around us, we sometimes wonder how everything we see and interact with came to be. One way to explain this is the theory of evolution. The theory of evolution suggests that the living organisms that we see today did not suddenly exist that way, but evolved through millions of years of subtle changes, with each generation adapting to its environment. This implies that the physical and cognitive characteristics of each living organism are a result of best fitting to its environment for survival. Evolution suggests that organisms evolve through reproduction by producing children of mixed genes from their parents. Given the fitness of

these individuals in their environment, stronger individuals have a higher likelihood of survival.

We often make the mistake of thinking that evolution is a linear process, with clear changes in successors. In reality, evolution is far more chaotic, with divergence in a species. A multitude of variants of a species are created through reproduction and mixing of genes. Noticeable differences in a species could take thousands of years to manifest and be realized only by comparing the average individual in each of those time points. Figure 4.1 depicts actual evolution versus the commonly mistaken version of the evolution of humans.

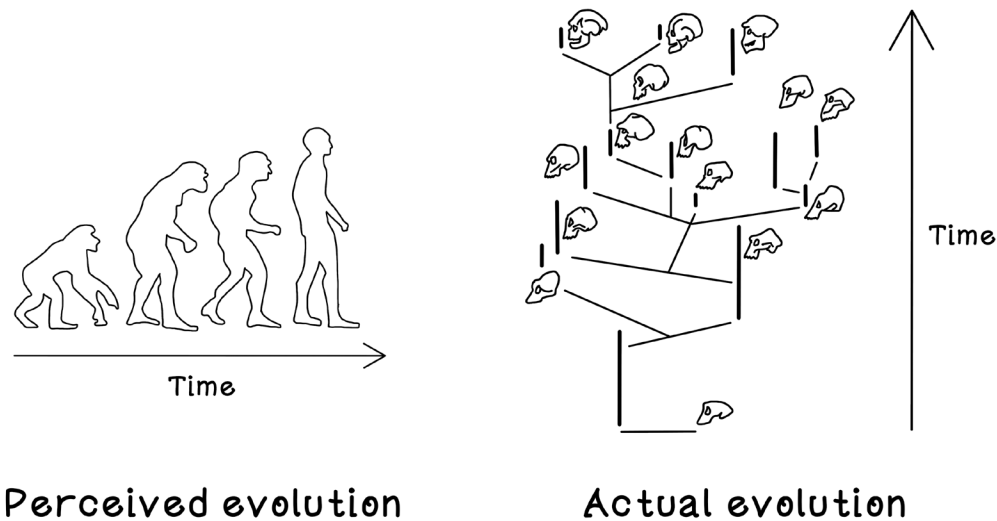


Figure 4.1 The idea of linear human evolution vs. actual human evolution

Charles Darwin proposed a theory of evolution that centers on natural selection. *Natural selection* is the concept that stronger members of a population are more likely to survive due to being more fit for their environment, which means they reproduce more and, thus, carry traits that are beneficial to survival to future generations—that could potentially perform better than their ancestors.

A classic example of evolution for adaption is the peppered moth. The peppered moth was originally light in color, which made for good camouflage against predators as the moth could blend in with light-colored surfaces in its environment. Only around 2% of the moth population was darker in color. After the Industrial Revolution, around 95% of the species were of the darker color variant. One explanation is that the lighter-colored moths could not blend in with as many surfaces anymore because pollution had darkened surfaces; thus lighter-colored moths were eaten more by predators because those moths were more visible. The darker moths had a greater advantage in blending in with

the darker surfaces, so they survived longer and reproduced more, and their genetic information was more widely spread to successors.

Among the peppered moths, the attribute that changed on a high level was the color of the moth. This property didn't just magically switch, however. For the change to happen, genes in moths with the darker color had to be carried to successors.

In other examples of natural evolution, we may see dramatic changes in more than simply color between different individuals, but in actuality, these changes are influenced by lower-level genetic differences over many generations (figure 4.2).

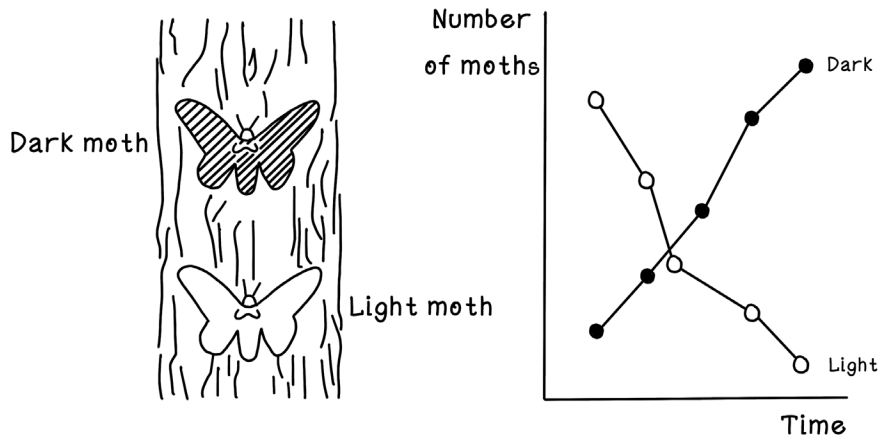


Figure 4.2 The evolution of the peppered moth

Evolution encompasses the idea that in a population of a species, pairs of organisms reproduce. The offspring are a combination of the parent's genes, but small changes are made in that offspring through a process called *mutation*. Then the offspring become part of the population. Not all members of a population live on, however. As we know, disease, injury, and other factors cause individuals to die. Individuals that are more adaptive to the environment around them are more likely to live on, a situation that gave rise to the term *survival of the fittest*. Based on Darwinian evolution theory, a population has the following attributes:

- *Variety*—Individuals in the population have different genetic traits.
- *Hereditary*—A child inherits genetic properties from its parents.
- *Selection*—A mechanism that measures the fitness of individuals. Stronger individuals have the highest likelihood of survival (survival of the fittest).

These properties imply that the following things happen during the process of evolution (figure 4.3):

- *Reproduction*—Usually, two individuals in the population reproduce to create offspring.
- *Crossover and mutation*—The offspring created through reproduction contain a mix of their parents' genes and have slight random changes in their genetic code.

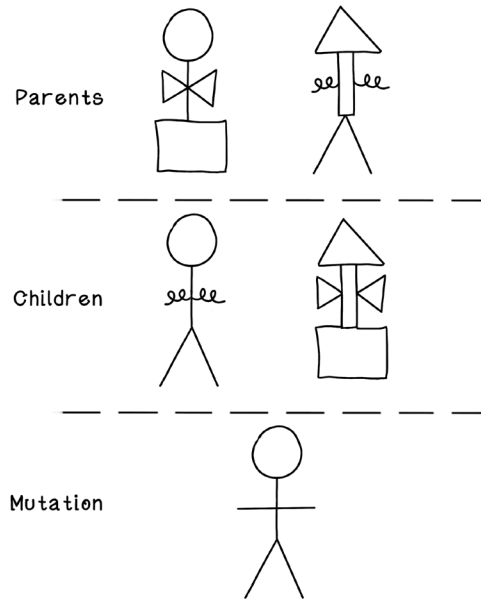


Figure 4.3 A simple example of reproduction and mutation

In summary, evolution is a marvelous and chaotic system that produces variations of life forms, some of which are better than others for specific things in specific environments. This theory also applies to evolutionary algorithms; learnings from biological evolution are harnessed for finding optimal solutions to practical problems by generating diverse solutions and converging on better-performing ones over many generations.

This chapter and chapter 5 are dedicated to exploring evolutionary algorithms, which are powerful but underrated approaches to solving hard problems. Evolutionary algorithms can be used in isolation or in conjunction with constructs such as neural networks. Having a solid grasp of this concept opens many possibilities for solving different novel problems.

Problems applicable to evolutionary algorithms

Evolutionary algorithms aren't applicable to solving all problems, but they are powerful for solving optimization problems in which the solution consists of a large number of permutations or choices. These problems typically consist of many valid solutions, with some being more optimal than others.

Consider the Knapsack Problem, a classic problem used in computer science to explore how algorithms work and how efficient they are. In the Knapsack Problem, a knapsack has a specific maximum weight that it can hold. Several items are available to be stored in the knapsack, and each item has a different weight and value. The goal is to fit as many items into the knapsack as possible so that the total value is maximized and the total weight does not exceed the knapsack's limit. The physical size and dimensions of the items are ignored in the simplest variation of the problem (figure 4.4).

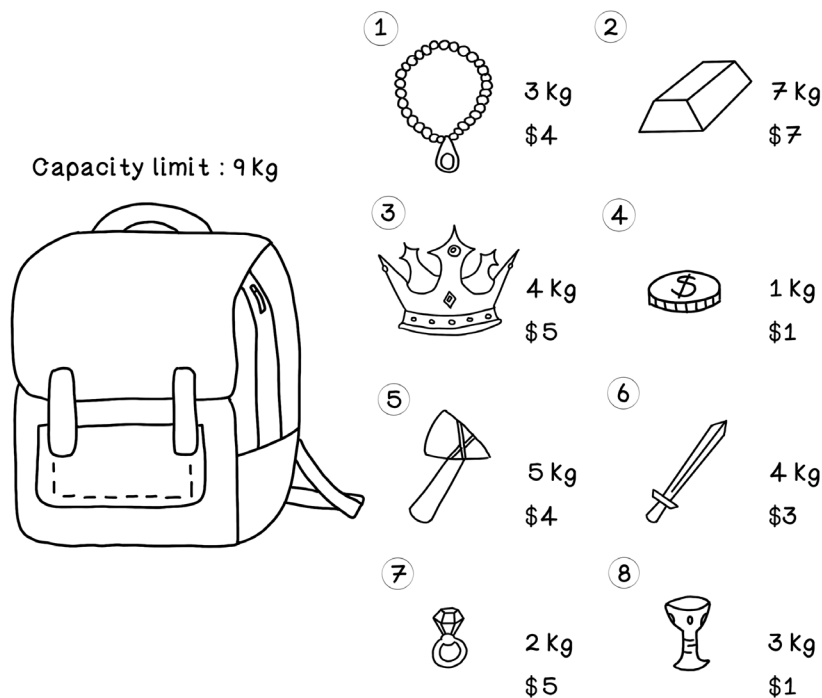


Figure 4.4 A simple Knapsack Problem example

As a trivial example, given the specification of the problem in table 4.1, a knapsack can hold a total weight capacity of 9 kg, and it could contain any of the eight items of varying weight and value.

Table 4.1 Knapsack weight capacity: 9 kg

Item ID	Item name	Weight (kg)	Value (\$)
1	Pearls	3	4
2	Gold	7	7
3	Crown	4	5
4	Coin	1	1
5	Axe	5	4
6	Sword	4	3
7	Ring	2	5
8	Cup	3	1

This problem has 255 possible solutions, including the following (figure 4.5):

- *Solution 1*—Include Item 1, Item 4, and Item 6. The total weight is 8 kg, and the total value is \$8.
- *Solution 2*—Include Item 1, Item 3, and Item 7. The total weight is 9 kg, and the total value is \$14.
- *Solution 3*—Include Item 2, Item 3, and Item 6. The total weight is 15 kg, which exceeds the knapsack's capacity.

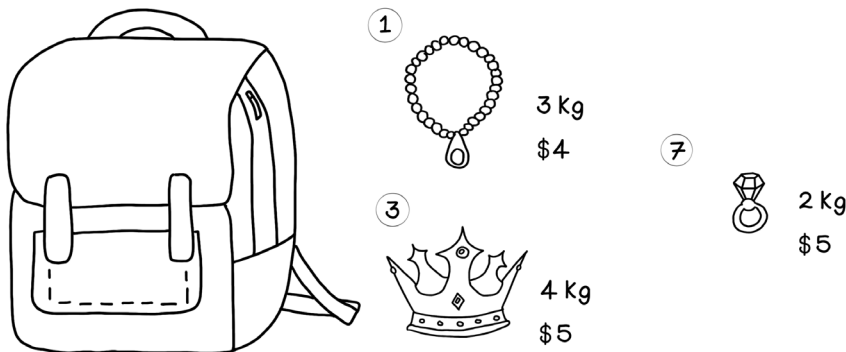


Figure 4.5 The optimal solution for the simple Knapsack Problem example

Clearly, the solution with the most value is *Solution 2*. Don't concern yourself too much about how the number of possibilities is calculated, but understand that the possibilities explode as the number of potential items increases.

Although this trivial example can be solved by hand, the Knapsack Problem could have varying weight constraints, a varying number of items, and varying weights and values for each item, making it impossible to solve by hand as the variables grow larger. It will also be computationally expensive to try to brute-force every combination of items when the variables grow; thus, we look for algorithms that are efficient at finding a desirable solution.

Note that we qualify the best solution we can find as a *desirable* solution rather than the *optimal* solution. Although some algorithms attempt to find the one true optimal solution to the Knapsack Problem, an evolutionary algorithm attempts to find the optimal solution but is not guaranteed to find it. The algorithm will find a solution that is acceptable for the use case, however—a subjective opinion of what an acceptable solution is, based on the problem. For a mission-critical health system, for example, a “good enough” solution may not cut it; but for a song-recommender system, it may be acceptable.

Now consider the larger dataset (yes, a giant knapsack) in table 4.2, in which the number of items and varying weights and values makes the problem difficult to solve by hand. By understanding the complexity of this dataset, you can easily see why many computer science algorithms are measured by their performance in solving such problems. Performance is defined as how well a specific solution solves a problem, not necessarily computational performance. In the Knapsack Problem, a solution that yields a higher total value would be better-performing. Evolutionary algorithms provide one method of finding solutions to the Knapsack Problem.

Table 4.2 Knapsack capacity: 6,404,180 kg

Item ID	Item name	Weight (kg)	Value (\$)
1	Axe	32,252	68,674
2	Bronze coin	225,790	471,010
3	Crown	468,164	944,620
4	Diamond statue	489,494	962,094
5	Emerald belt	35,384	78,344
6	Fossil	265,590	579,152
7	Gold coin	497,911	902,698
8	Helmet	800,493	1,686,515
9	Ink	823,576	1,688,691
10	Jewel box	552,202	1,056,157
11	Knife	323,618	677,562
12	Long sword	382,846	833,132
13	Mask	44,676	99,192
14	Necklace	169,738	376,418
15	Opal badge	610,876	1,253,986
16	Pearls	854,190	1,853,562
17	Quiver	671,123	1,320,297
18	Ruby ring	698,180	1,301,637
19	Silver bracelet	446,517	859,835
20	Timepiece	909,620	1,677,534
21	Uniform	904,818	1,910,501
22	Venom potion	730,061	1,528,646
23	Wool scarf	931,932	1,827,477
24	Crossbow	952,360	2,068,204
25	Yesteryear book	926,023	1,746,556
26	Zinc cup	978,724	2,100,851

One way to solve this problem is to use a brute-force approach. This approach involves calculating every possible combination of items and determining the value of each combination that satisfies the knapsack's weight constraint until the best solution is encountered.

Figure 4.6 shows some benchmark analytics for the brute-force approach. Note that the computation is based on the hardware of an average personal computer.

Combinations	$2^{26} = 67,108,864$
Iterations	$2^{26} = 67,108,864$
Accuracy	100%
Compute time	~7 minutes

Figure 4.6 Performance analytics of brute-forcing the Knapsack Problem

Keep the Knapsack Problem in mind, as it will be used throughout this chapter as we attempt to understand, design, and develop a genetic algorithm to find acceptable solutions to this problem.

NOTE A note about the term *performance*: From the perspective of an individual solution, performance is how well the solution solves the problem. From the perspective of the algorithm, performance may be how well a specific configuration does in finding a solution. Finally, performance may mean computational cycles. Bear in mind that this term is used differently based on the context.

The thinking behind using a genetic algorithm to solve the Knapsack Problem can be applied to a range of practical problems. If a logistics company wants to optimize the packing of trucks based on their destinations, for example, a genetic algorithm would be useful. If that same company wanted to find the shortest route between several destinations, a genetic algorithm would be useful as well. If a factory refined items into raw material via a conveyor-belt system, and the order of the items influenced productivity, a genetic algorithm would be useful in determining that order.

When we dive into the thinking, approach, and life cycle of the genetic algorithm, it should become clear where this powerful algorithm can be applied, and perhaps you will think of other uses in your work. It is important to keep in mind that a genetic algorithm is *stochastic*, which means that the output of the algorithm is likely to be different each time it is run.

Genetic algorithm: Life cycle

The genetic algorithm is a specific algorithm in the family of evolutionary algorithms. Each algorithm works on the same premise of evolution but has small tweaks in the different parts of the life cycle to cater to different problems. We explore some of these parameters in chapter 5.

Genetic algorithms are used to evaluate large search spaces for a good solution. It is important to note that a genetic algorithm is not guaranteed to find the absolute best solution; it attempts to find the global best while avoiding local best solutions.

A *global best* is the best possible solution, and a *local best* is a solution that is less optimal. Figure 4.7 represents the possible best solutions if the solution must be minimized—that is, the smaller the value, the better. If the goal was to maximize a solution, the larger the value, the better. Optimization algorithms like genetic algorithms aim to incrementally find local best solutions in search of the global best solution.

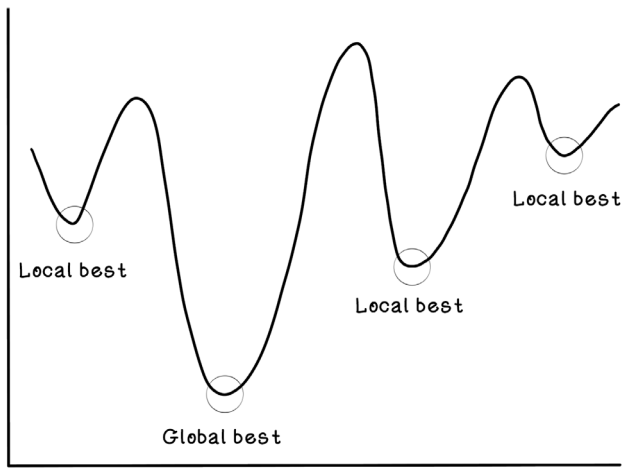


Figure 4.7 Local best vs. global best

Careful attention is needed when configuring the parameters of the algorithm so that it strives for diversity in solutions at the start and gradually gravitates toward better solutions through each generation. At the start, potential solutions should vary widely in individual genetic attributes. Without divergence at the start, the risk of getting stuck in a local best increases (figure 4.8).

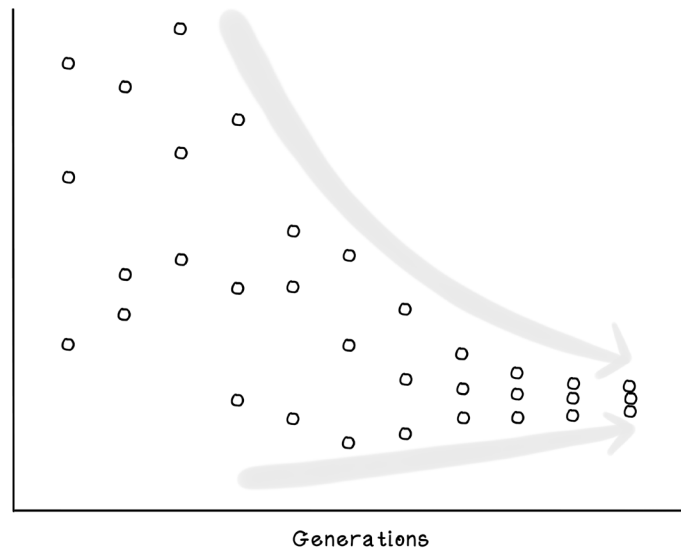


Figure 4.8 Diversity to convergence

The configuration for a genetic algorithm is based on the problem space. Each problem has a unique context and a different domain in which data is represented, and solutions are evaluated differently.

The general life cycle of a genetic algorithm is as follows:

- *Creating a population*—Creating a random population of potential solutions.
- *Measuring the fitness of individuals in the population*—Determining how good a specific solution is. This task is accomplished by using a fitness function that scores solutions to determine how good they are.
- *Selecting parents based on their fitness*—Selecting pairs of parents that will reproduce offspring.
- *Reproducing individuals from parents*—Creating offspring from their parents by mixing genetic information and applying slight mutations to the offspring.
- *Populating the next generation*—Selecting individuals and offspring from the population that will survive to the next generation.

Several steps are involved in implementing a genetic algorithm. These steps encompass the stages of the algorithm life cycle (figure 4.9).

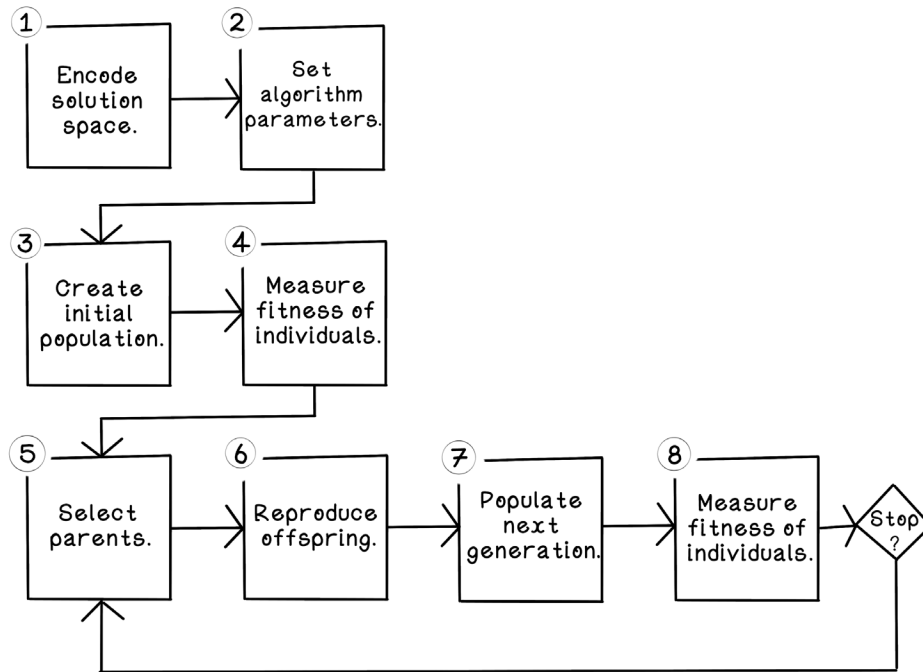


Figure 4.9 Genetic algorithm life cycle

With the Knapsack Problem in mind, how would we use a genetic algorithm to find solutions to the problem? The next section dives into the process.

Encoding the solution spaces

When we use a genetic algorithm, it is paramount to do the encoding step correctly, which requires careful design of the representation of possible states. The *state* is a data structure with specific rules that represents possible solutions to a problem. Furthermore, a collection of states forms a population (figure 4.10).

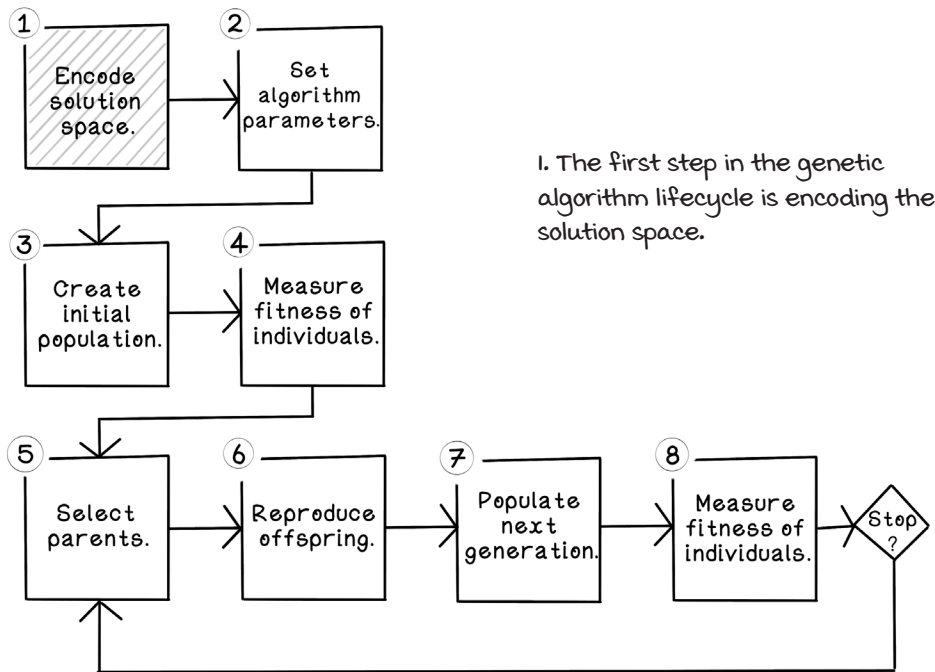


Figure 4.10 Encode the solution.

Terminology

With respect to evolutionary algorithms, an individual candidate solution is called a *chromosome*. A chromosome is made up of genes. The *gene* is the logical type for the unit, and the *allele* is the actual value stored in that unit. A *genotype* is a representation of a solution, and a *phenotype* is a unique solution itself. Each chromosome always has the same number of genes. A collection of chromosomes forms a *population* (figure 4.11).

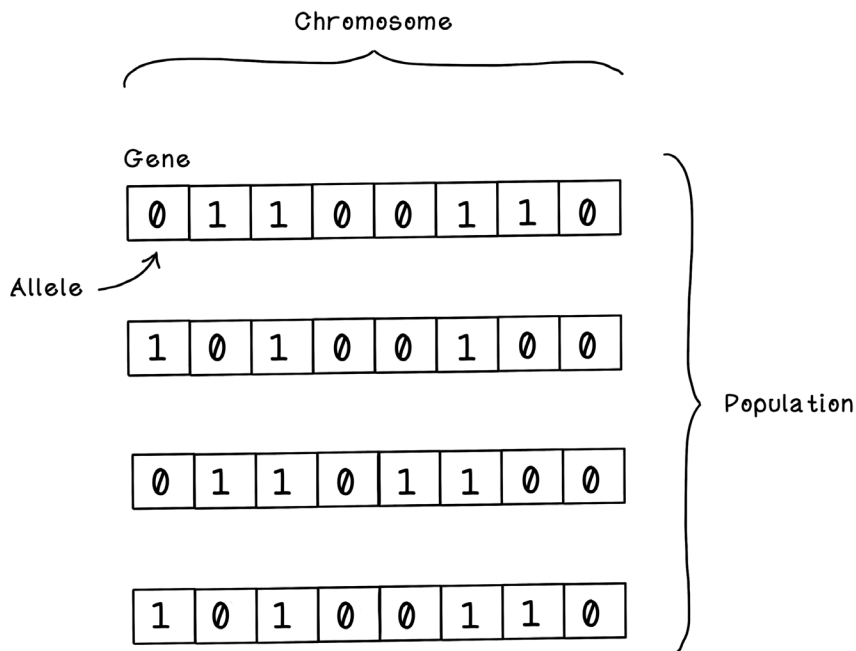


Figure 4.11 Terminology of the data structures representing a population of solutions

In the Knapsack Problem, several items can be placed in the knapsack. A simple way to describe a possible solution that contains some items but not others is binary encoding (figure 4.12). *Binary encoding* represents excluded items with 0s and included items with 1s. If the value at gene index 3 is 1, for example, that item is marked to be included. The complete binary string is always the same size: the number of items available for selection. Several alternative encoding schemes exist, however, and are described in chapter 5.

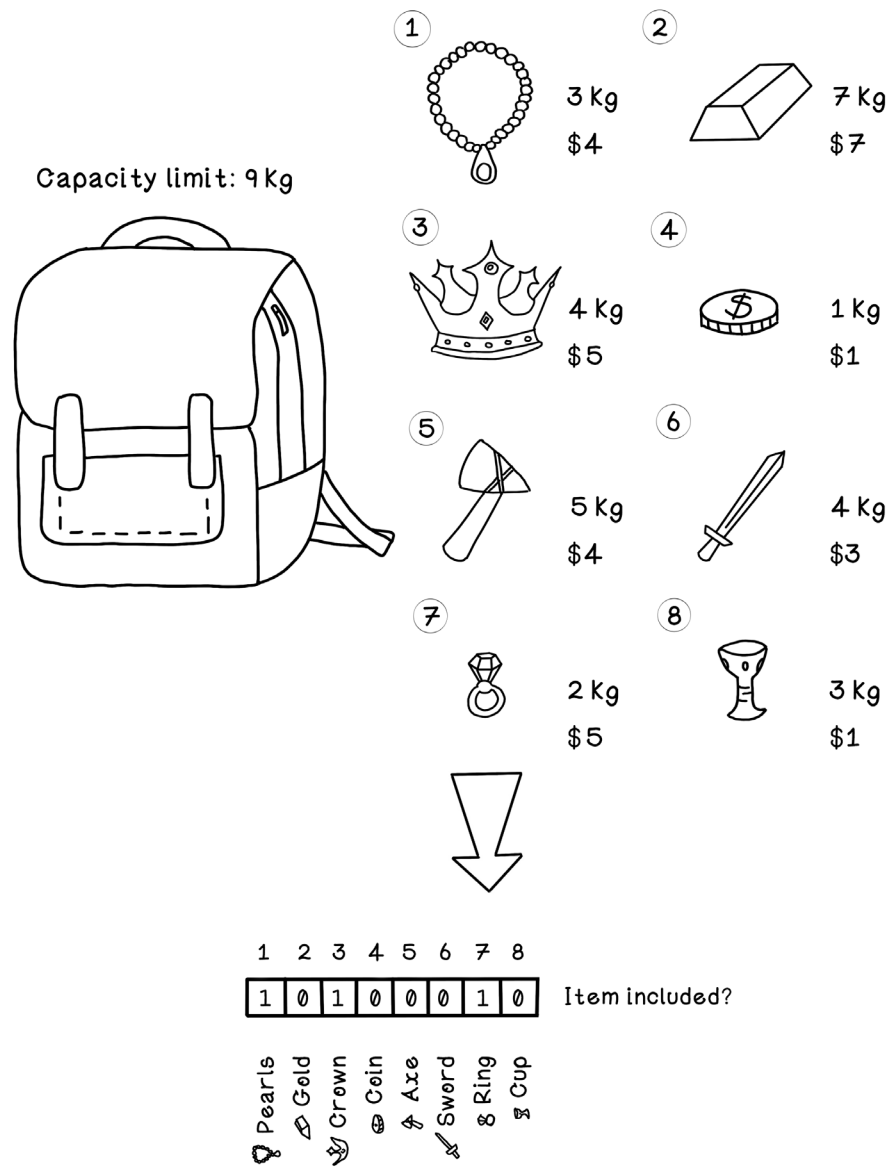


Figure 4.12 Binary-encoding the Knapsack Problem

Binary encoding: Representing possible solutions with zeros and ones

Binary encoding represents a gene in terms of 0 or 1, so a chromosome is represented by a string of binary bits. Binary encoding can be used in versatile ways to express the presence of a specific element or even encoding numeric values as binary numbers. The advantage of binary encoding is that it is usually more performant due to the use of primitive types. Using binary encoding places less demand on working memory, and depending on the language used, binary operations are computationally faster. But critical thought must be used to ensure that the encoding makes sense for the respective problem and represents potential solutions well; otherwise, the algorithm may perform poorly (figure 4.13).

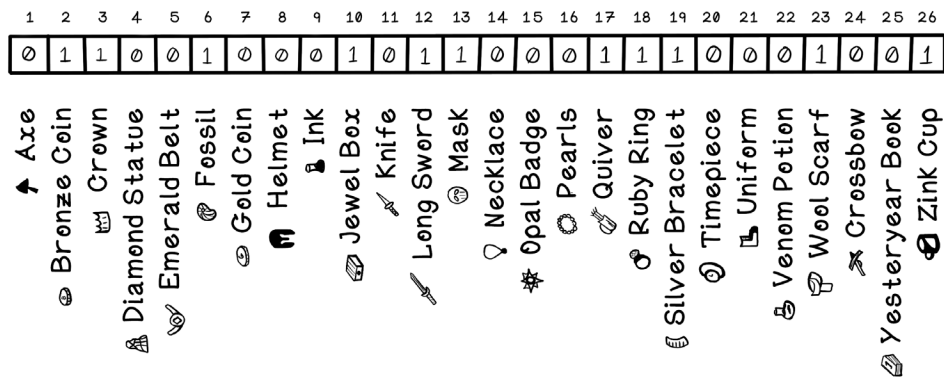


Figure 4.13 Binary-encoding the larger dataset for the Knapsack Problem

Given the Knapsack Problem with a dataset that consists of 26 items of varying weight and value, a binary string can be used to represent the inclusion of each item. The result is a 26-character string in which for each index, 0 means that the respective item is excluded and 1 means that the respective item is included.

Other encoding schemes—including real-value encoding, order encoding, and tree encoding—are discussed in chapter 5.

EXERCISE: WHAT IS A POSSIBLE ENCODING FOR THE FOLLOWING PROBLEM?

Suppose we have the following sentence and want to find which words can be excluded or included to maintain a meaningful phrase using a genetic algorithm:

THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG

Incorrect phrases

THE BROWN JUMPS OVER
 QUICK FOX OVER THE
THE FOX THE LAZY

Correct phrases

THE QUICK FOX
 QUICK FOX JUMPS
THE BROWN FOX DOG
THE BROWN LAZY DOG
THE QUICK DOG
 QUICK OVER THE DOG
THE QUICK LAZY DOG

*Punctuation is excluded.

SOLUTION: WHAT IS A POSSIBLE ENCODING FOR THE FOLLOWING PROBLEM?

Because the number of possible words is always the same, and the words are always in the same position, binary encoding can be used to describe which words are included and which are excluded. The chromosome consists of 9 genes, each gene indicating a word in the phrase.

THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG								
THE BROWN JUMPS OVER								
1	0	1	0	1	1	0	0	0
THE BROWN LAZY DOG								
1	0	1	0	0	0	0	1	1

Creating a population of solutions

In the beginning, the population was created. The first step in a genetic algorithm is initializing random potential solutions to the problem at hand. In the process of initializing the population, although the chromosomes are generated randomly, the constraints of the problem must be taken into consideration, and the potential solutions should be valid or assigned a terrible fitness score if they violate the constraints. Each individual in the population may not solve the problem well, but the solution is valid. As mentioned in the earlier example of packing items into a knapsack, a solution that specifies packing the same item more than once should be an invalid solution and should not form part of the population of potential solutions (figure 4.14).

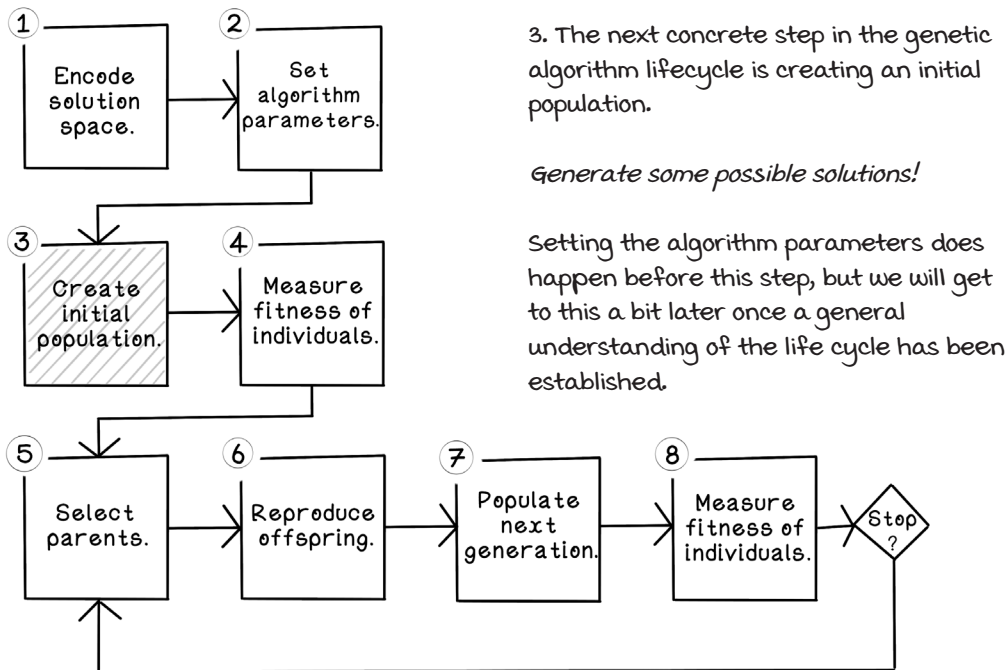


Figure 4.14 Create an initial population.

Given how the Knapsack Problem's solution state is represented, this implementation randomly decides whether each item should be included in the bag. That said, only solutions that satisfy the weight-limit constraint should be considered. The problem with simply moving from left to right and randomly choosing whether the item is included is that it creates a bias toward the items on the left end of the chromosome. Similarly, if we start from the right, we will be biased toward items on the right. One possible way to get

around this is to generate an entire individual with random genes and then determine whether the solution is valid and does not violate any constraints. Assigning a terrible score to invalid solutions can solve this problem (figure 4.15).

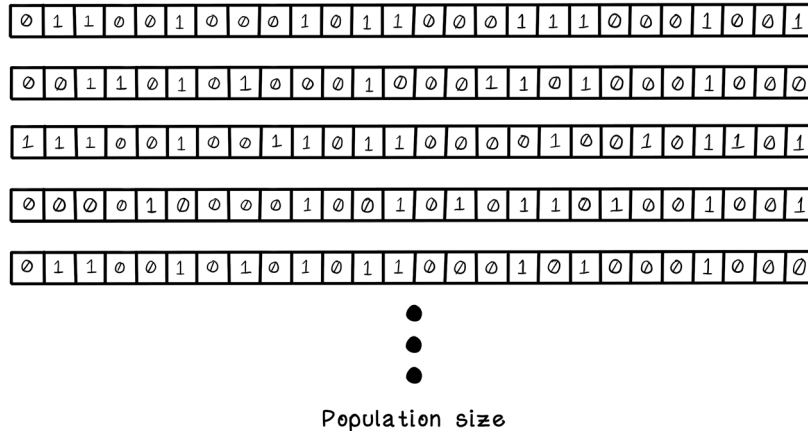


Figure 4.15 An example of a population of solutions

Pseudocode

To generate an initial population of possible solutions, an empty array is created to hold the individuals. Then, for each individual in the population, an empty array is created to hold the genes of the individual. Each gene is randomly set to 1 or 0, indicating whether the item at that gene index is included:

```
generate_initial_population (population_size, individual_size)
  let population be an empty array
  for individual in range 0 to population_size
    let current_individual be an empty array
    for gene in range 0 to individual_size
      let random_gene be 0 or 1 randomly
      append random_gene to current_individual
    append current_individual to population
  return population
```

Measuring fitness of individuals in a population

When a population has been created, the fitness of each individual in the population needs to be determined. Fitness defines how well a solution performs. The fitness function is critical to the life cycle of a genetic algorithm. If the fitness of the individuals is measured incorrectly or in a way that does not attempt to strive for the optimal solution, the selection process for parents of new individuals and new generations will be influenced; the algorithm will be flawed and cannot strive to find the best possible solution.

Fitness functions are similar to the heuristics that we explored in chapter 3. They are guidelines for finding good solutions (figure 4.16).

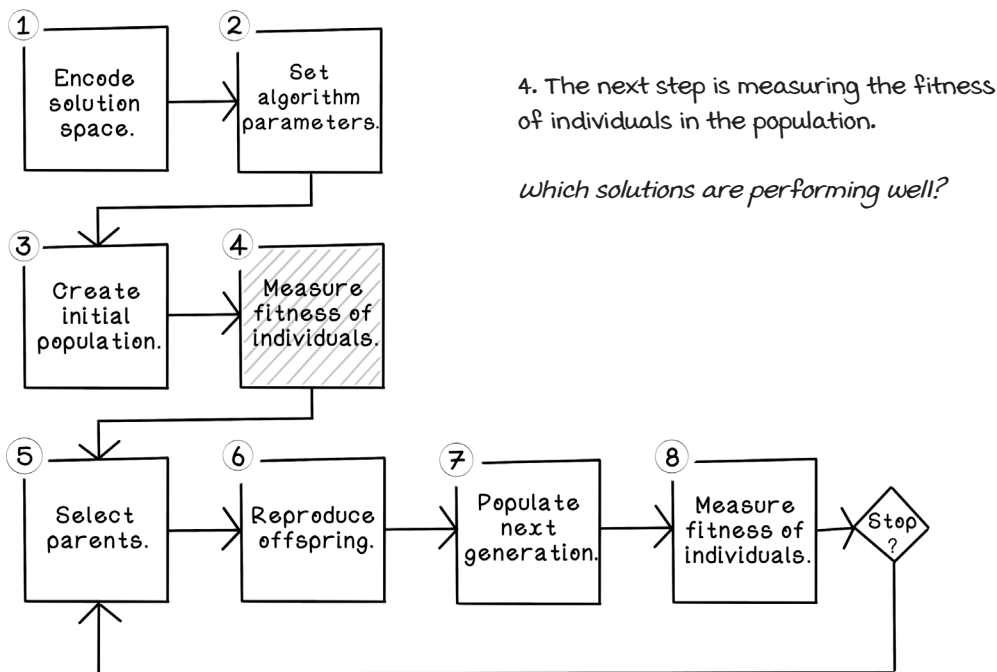


Figure 4.16 Measure the fitness of individuals.

In our example, the solution attempts to maximize the value of the items in the knapsack while respecting the weight-limit constraints. The fitness function measures the total value of the items in the knapsack for each individual. The result is that individuals with higher total values are more fit. Note that an invalid individual appears in figure 4.17, to highlight that its fitness score would result in 0—a terrible score, because it exceeds the weight capacity for this instance of the problem, which is 6,404,180.

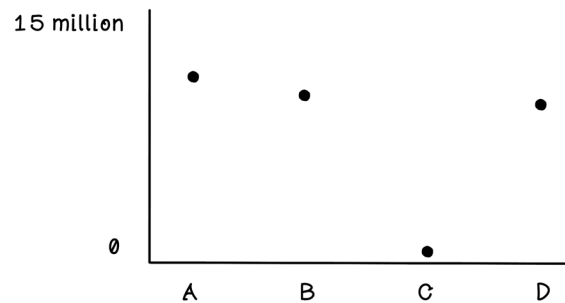
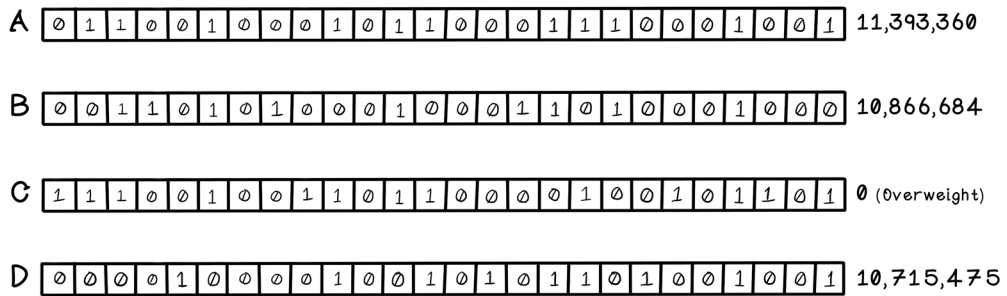


Figure 4.17 Measuring the fitness of individuals

Depending on the problem being solved, the result of the fitness function may be required to be minimized or maximized. In the Knapsack Problem, the contents of the knapsack can be maximized within constraints, or the empty space in the knapsack could be minimized. The approach depends on the interpretation of the problem.

Pseudocode

To calculate the fitness of an individual in the Knapsack Problem, the sums of the values of each item that the respective individual includes must be determined. This task is accomplished by setting the total value to 0 and then iterating over each gene to determine whether the item it represents is included. If the item is included, the value of the item represented by that gene is added to the total value. Similarly, the total weight is calculated to ensure that the solution is valid. The concepts of calculating fitness and checking constraints can be split for clearer separation of concerns:

```

calculate_individual_fitness (individual,
                               knapsack_items,
                               knapsack_max_weight)

  let total_weight equal 0
  let total_value equal 0
  for gene_index in range 0 to length of individual
    let current_bit equal individual[gene_index]
    if current_bit equals 1
      add weight of knapsack_items[gene_index] to total_weight
      add value of knapsack_items[gene_index] to total_value
  if total_weight is greater than knapsack_max_weight
    return value as 0 since it exceeds the weight constraint
  return total_value as individual fitness

```

Selecting parents based on their fitness

The next step in a genetic algorithm is selecting parents that will produce new individuals. In Darwinian theory, the individuals that are more fit have a higher likelihood of reproduction than others because they typically live longer. Furthermore, these individuals contain desirable attributes for inheritance due to their superior performance in their environment. That said, some individuals are likely to reproduce even if they are not the fittest in the entire group, and these individuals may contain strong traits even though they are not strong in their entirety.

Each individual has a calculated fitness that is used to determine the probability of it being selected to be a parent to a new individual. This attribute makes the genetic algorithm stochastic in nature (figure 4.18).

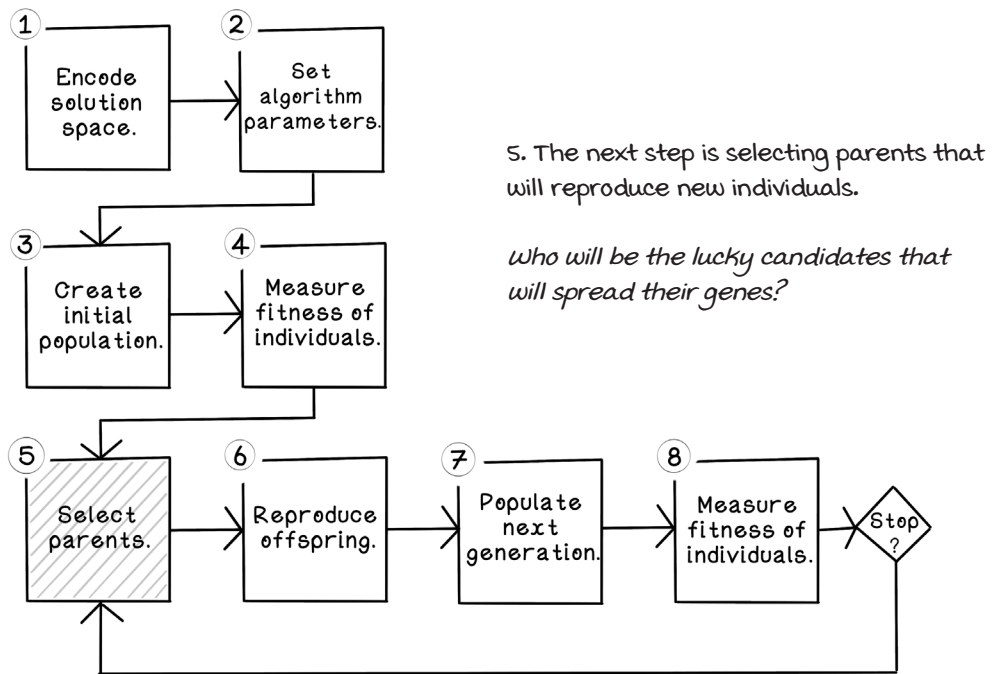


Figure 4.18 Select parents.

A popular technique in choosing parents based on their fitness is *roulette-wheel selection*. This strategy gives different individuals portions of a wheel based on their fitness. The wheel is “spun,” and an individual is selected. Higher fitness gives an individual a larger slice of the wheel. This process is repeated until the desired number of parents is reached.

By calculating the probabilities of 16 individuals of varying fitness, the wheel allocates a slice to each. Because many individuals perform similarly, there are many slices of similar size (figure 4.19).

A	1 0 1 1 1 0 0 1 1 0 1 1 0 1 0 0 0 1 1 0 0 1 0 0 0 1	13,107,019
B	1 1 0 0 0 1 0 0 1 1 1 1 1 1 1 1 0 0 0 1 0 0 0	12,965,145
C	0 0 1 1 0 1 1 0 1 0 0 1 1 0 0 0 0 1 0 1 0 1 1 0 0 0	12,344,873
D	0 0 1 1 1 1 1 0 0 1 1 0 0 1 1 0 1 0 0 1 1 0 0 0 0 0	11,739,363
E	1 1 0 0 0 1 0 0 0 1 1 1 1 1 1 0 1 1 0 1 0 0 0 1 0 0 0	11,711,159
F	1 1 0 0 0 1 0 0 1 1 1 1 1 0 1 0 1 1 0 1 0 0 0 1 0 0 0	11,611,967
G	1 0 1 0 0 1 1 1 0 0 0 0 1 0 0 1 0 1 1 0 0 0 0 0 0 1 0	10,042,441
H	1 1 0 0 0 1 0 0 1 1 1 1 1 1 1 0 1 1 1 0 1 0 0 0 0 0 0	9,883,682
I	1 1 0 0 0 1 0 0 1 1 1 1 1 1 1 0 0 1 0 1 0 0 0 1 0 0 0	9,857,597
J	0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 1 0 1 0 0 1	9,670,184
K	0 0 0 0 1 1 0 1 1 1 0 1 0 1 0 0 0 1 0 1 0 0 0 0 0 0 0	9,277,580
L	1 0 0 0 0 1 0 0 1 0 0 0 0 0 1 0 0 1 1 0 0 0 1 0 1 0 0	8,931,719
M	0 1 0 0 0 0 0 0 1 1 1 1 0 1 1 1 1 0 0 0 1 0 0 0 0 0 0	8,324,936
N	1 1 1 1 0 0 1 0 0 0 1 0 1 0 0 0 0 0 0 1 1 0 1 0 0 0 0	8,018,760
O	0 0 0 1 1 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 1 0 0 1	6,900,314
P	0 0 0 1 1 0 0 0 0 0 0 1 1 0 1 0 0 0 1 0 0 0 1 0 0 0	6,056,664

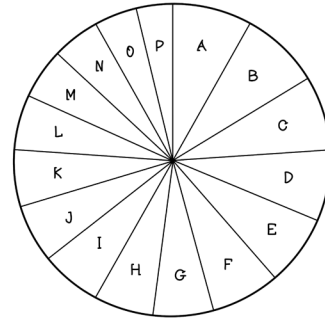


Figure 4.19 Determining the probability of selection for each individual

The number of parents selected to be used for reproducing new offspring is determined by the intended total number of offspring required, which is determined by the desired population size for each generation. Two parents are selected, and offspring are created. This process repeats with different parents selected (with a chance of the same individuals being a parent more than once) until the desired number of offspring have been generated. Two parents can reproduce a single mixed child or two mixed children. This concept will be made clearer later in this chapter. In our Knapsack Problem example, the individuals with greater fitness are those that fill the bag with the most combined value while respecting the weight-limit constraint.

Population models are ways to control the diversity of the population. Steady state and generational are two population models that have their own advantages and disadvantages.

Steady state: Replacing a portion of the population each generation

This high-level approach to population management is not an alternative to the other selection strategies, but a scheme that uses them. The idea is that the majority of the population is retained, and a small group of weaker individuals are removed and replaced with new offspring. This process mimics the cycle of life and death, in which weaker

individuals die and new individuals are made through reproduction. If there were 100 individuals in the population, a portion of the population would be existing individuals, and a smaller portion would be new individuals created via reproduction. There may be 80 individuals from the current generation and 20 new individuals.

Generational: Replacing the entire population each generation

This high-level approach to population management is similar to the steady-state model but is not an alternative to selection strategies. The generational model creates a number of offspring individuals equal to the population size and replaces the entire population with the new offspring. If there were 100 individuals in the population, each generation would result in 100 new individuals via reproduction. Steady state and generational are overarching ideas for designing the configuration of the algorithm.

Roulette wheel: Selecting parents and surviving individuals

Chromosomes with higher fitness scores are more likely to be selected, but chromosomes with lower fitness scores still have a small chance of being selected. The term *roulette-wheel selection* comes from a roulette wheel at a casino, which is divided into slices. Typically, the wheel is spun, and a marble is released into the wheel. The selected slice is the one that the marble lands on when the wheel stops turning.

In this analogy, chromosomes are assigned to slices of the wheel. Chromosomes with higher fitness scores have larger slices of the wheel, and chromosomes with lower fitness scores have smaller slices. A chromosome is selected randomly, much as a ball randomly lands on a slice.

This analogy is an example of probabilistic selection. Each individual has a chance of being selected, whether that chance is small or high. The chance of selection of individuals influences the diversity of the population and convergence rates mentioned earlier in this chapter. Figure 4.19, also earlier in this chapter, illustrates this concept.

Pseudocode

First, the probability of selection for each individual needs to be determined. This probability is calculated for each individual by dividing its fitness by the total fitness of the population. Roulette-wheel selection can be used. The “wheel” is “spun” until the desired number of individuals have been selected. For each selection, a random decimal number between 0 and 1 is calculated. If an individual’s fitness is within that probability, it is selected. Other probabilistic approaches may be used to determine the probability of each individual, including standard deviation, in which an individual’s value is compared with the mean value of the group:

```

set_probabilities_of_population (population)
    let total_fitness equal the sum of fitness of the population
    for individual in population
        let the probability_of_selection of individual...
            ...equal it's fitness/total_fitness

roulette_wheel_selection(population, number_of_selections):
    let possible_probabilities equal
        set_probabilities_of_population (population)
    let slices equal empty array
    let total equal 0
    for i in range(0, number_of_selections):
        append [i, total, total + possible_probabilities[i]]
            to slices
        total += possible_probabilities[i]
    let spin equal random(0, 1)
    let result equal [slice for slice in slices if slice[1] < spin <= slice[2]]
    return result

```

Reproducing individuals from parents

When parents are selected, reproduction needs to happen to create new offspring from the parents. Generally, two steps are related to creating children from two parents. The first concept is *crossover*, which means mixing part of the chromosome of the first parent with part of the chromosome of the second parent, and vice versa. This process results in two offspring that contain inversed mixes of their parents. The second concept is *mutation*, which means randomly changing the offspring slightly to create variation in the population (figure 4.20).

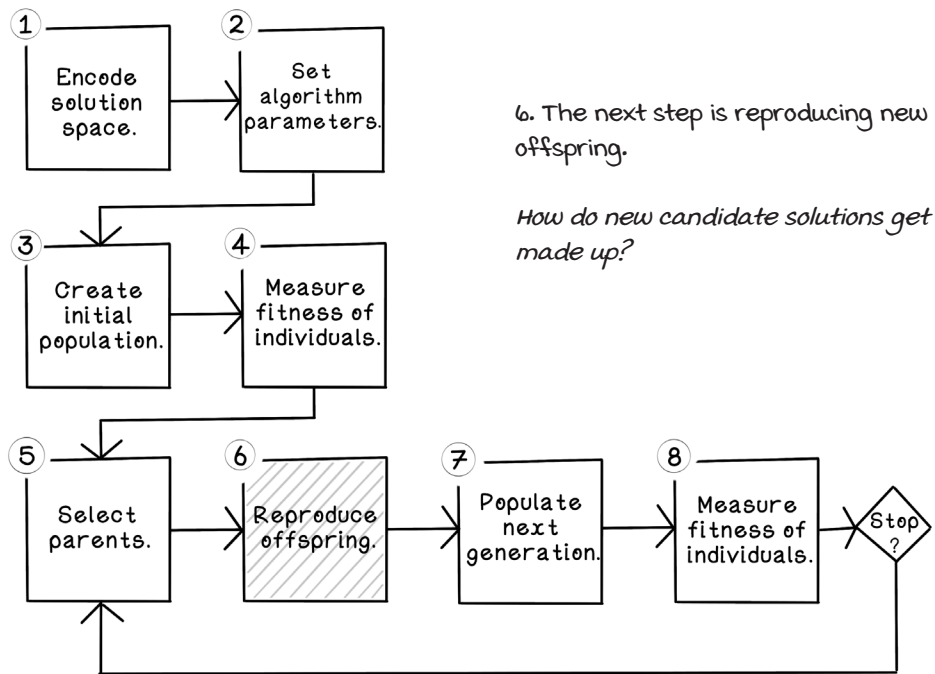


Figure 4.20 Reproduce offspring.

Crossover

Crossover involves mixing genes between two individuals to create one or more offspring individuals. Crossover is inspired by the concept of reproduction. The offspring individuals are parts of their parents, depending on the crossover strategy used. The crossover strategy is highly affected by the encoding used.

Single-point crossover: Inheriting one part from each parent

One point in the chromosome structure is selected. Then, by referencing the two parents in question, the first part of the first parent is used, and the second part of the second parent is used. These two parts combined create a new offspring. A second offspring can be made by using the first part of the second parent and the second part of the first parent.

Single-point crossover is applicable to binary encoding, order/permutation encoding, and real-value encoding (figure 4.21). These encoding schemes are discussed in chapter 5.

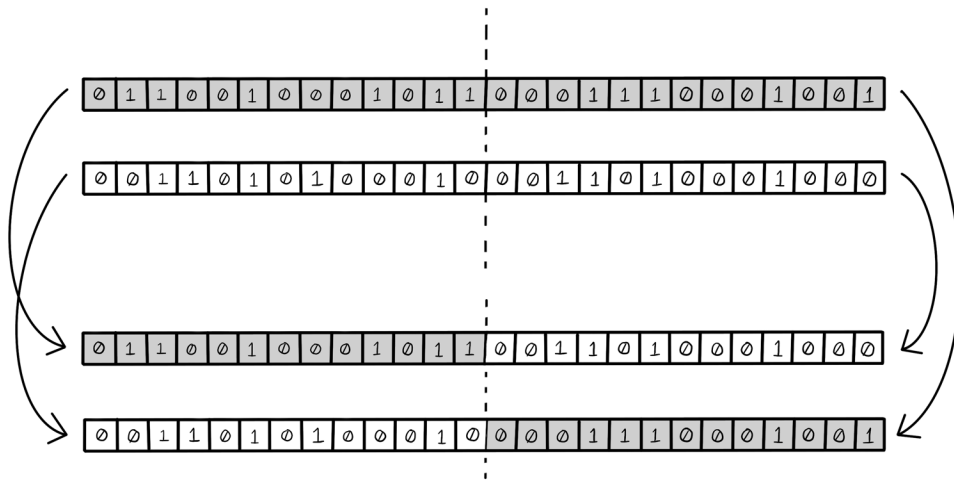


Figure 4.21 Single-point crossover

Pseudocode

To create two new offspring individuals, an empty array is created to hold the new individuals. All genes from index 0 to the desired index of parent A are concatenated with all genes from the desired index to the end of the chromosome of parent B, creating one offspring individual. The inverse creates the second offspring individual:

```
one_point_crossover (parent_a, parent_b, xover_point)
    let children equal empty array

    let child_1 equal genes 0 to xover_point from parent_a plus...
    ...genes xover_point to parent_b length from parent_b
    append child_1 to children

    let child_2 equal genes 0 to xover_point from parent_b plus...
    ...genes xover_point to parent_a length from parent_a
    append child_2 to children

    return children
```

Two-point crossover: Inheriting more parts from each parent

Two points in the chromosome structure are selected; then, referencing the two parents in question, parts are chosen in an alternating manner to make a complete offspring individual. This process is similar to single-point crossover, discussed earlier. To describe the process completely, the offspring consist of the first part of the first parent, the second part of the second parent, and the third part of the first parent. Think about two-point crossover as splicing arrays to create new ones. Again, a second individual can be made by using the inverse parts of each parent. Two-point crossover is applicable to binary encoding and real-value encoding (figure 4.22).

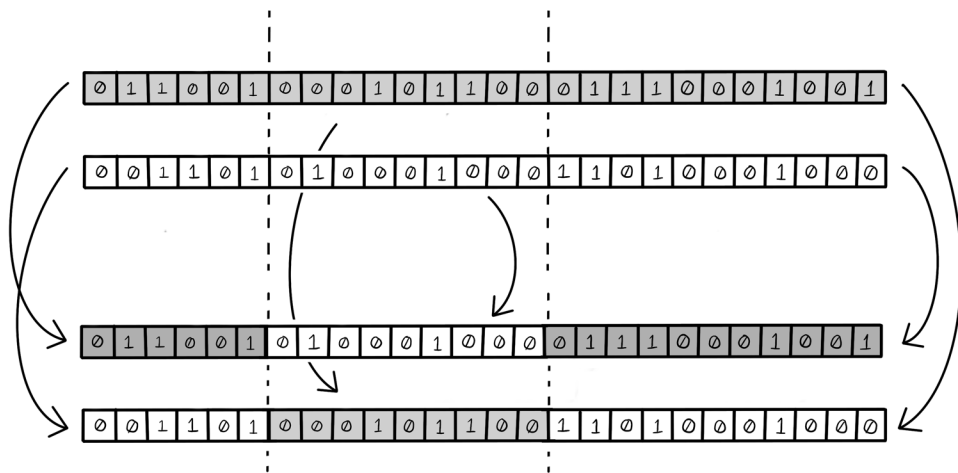


Figure 4.22 Two-point crossover

Uniform crossover: Inheriting many parts from each parent

Uniform crossover is a step beyond two-point crossover. In uniform crossover, a mask is created that represents which genes from each parent will be used to generate the child offspring. The inverse process can be used to make a second offspring. The mask can be generated randomly each time offspring are created to maximize diversity. Generally speaking, uniform crossover creates more-diverse individuals because the attributes of the offspring are quite different compared with any of their parents. Uniform crossover is applicable to binary encoding and real-value encoding (figure 4.23).

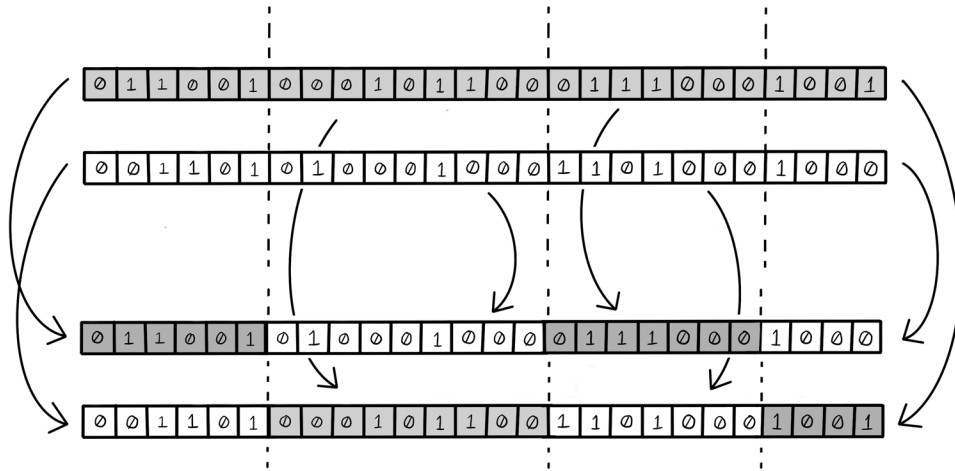


Figure 4.23 Uniform crossover

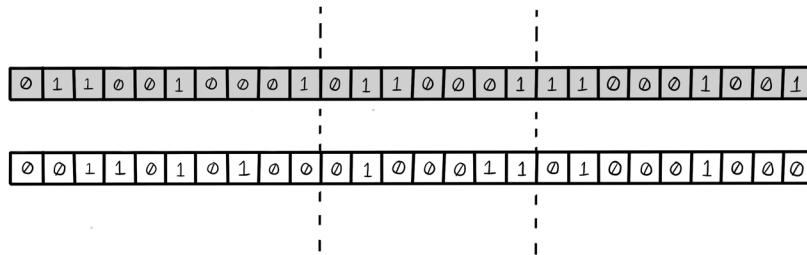
Mutation

Mutation involves changing offspring individuals slightly to encourage diversity in the population. Several approaches to mutation are used based on the nature of the problem and the encoding method.

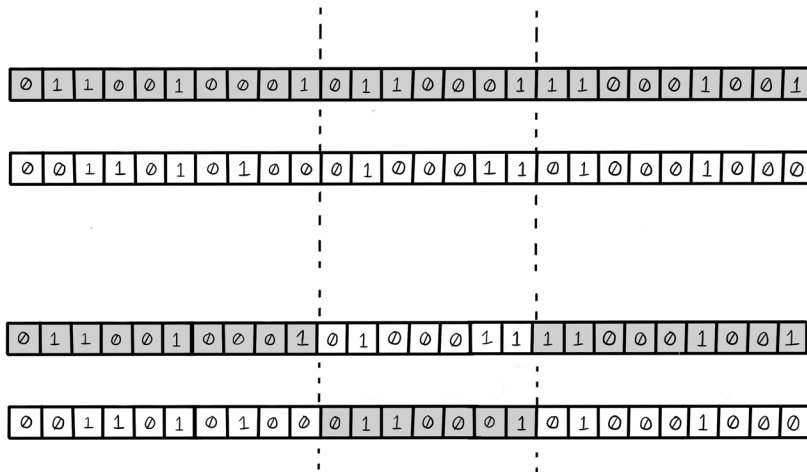
One parameter in mutation is the mutation rate—the likelihood that an offspring chromosome will be mutated. Similarly to living organisms, some chromosomes are mutated more than others; an offspring is not an exact combination of its parents' chromosomes but contains minor genetic differences. Mutation can be critical to encouraging diversity in a population and preventing the algorithm from getting stuck in local best solutions.

A high mutation rate means that individuals have a high chance of being selected to be mutated or that genes in the chromosome of an individual have a high chance of being mutated, depending on the mutation strategy. High mutation means more diversity, but too much diversity may result in the deterioration of good solutions.

EXERCISE: WHAT OUTCOME WOULD UNIFORM CROSSOVER GENERATE FOR THESE CHROMOSOMES?



SOLUTION: WHAT OUTCOME WOULD UNIFORM CROSSOVER GENERATE FOR THESE CHROMOSOMES?



Bit-string mutation for binary encoding

In bit-string mutation, a gene in a binary-encoded chromosome is selected randomly and changed to another valid value (figure 4.24). Other mutation mechanisms are applicable when nonbinary encoding is used. The topic of mutation mechanisms will be explored in chapter 5.

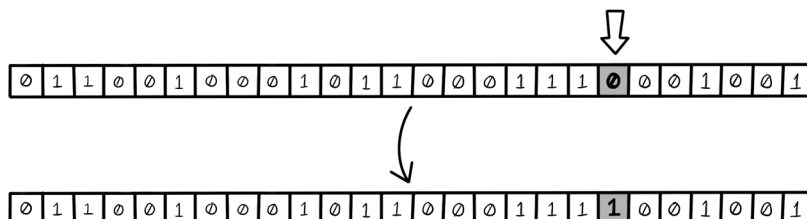


Figure 4.24 Bit-string mutation

Pseudocode

To mutate a single gene of an individual's chromosome, a random gene index is selected. If that gene represents 1, change it to represent 0, and vice versa:

```
mutate_individual (individual, chromosome_length)
  let random_index equal a random number between 0 and chromosome_length
  if gene at index random_index of individual is equal to 1:
    let gene at index random_index of individual equal 0
  else:
    let gene at index random_index of individual equal 1
  return individual
```

Flip-bit mutation for binary encoding

In flip-bit mutation, all genes in a binary-encoded chromosome are inverted to the opposite value. Where there were 1s are 0s, and where there were 0s are 1s. This type of mutation could degrade good-performing solutions dramatically and usually is used when diversity needs to be introduced into the population constantly (figure 4.25).

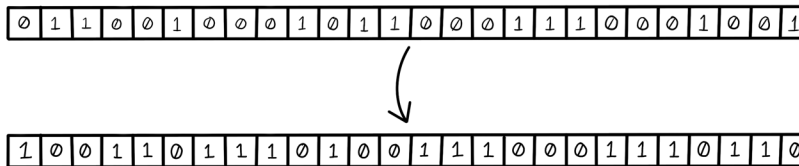


Figure 4.25 Flip-bit mutation

Populating the next generation

When the fitness of the individuals in the population has been measured and offspring have been reproduced, the next step is selecting which individuals live on to the next generation. The size of the population is usually fixed, and because more individuals have been introduced through reproduction, some individuals must die off and be removed from the population.

It may seem like a good idea to take the top individuals that fit into the population size and eliminate the rest. This strategy, however, could create stagnation in the diversity of individuals if the individuals that survive are similar in genetic makeup (figure 4.26).

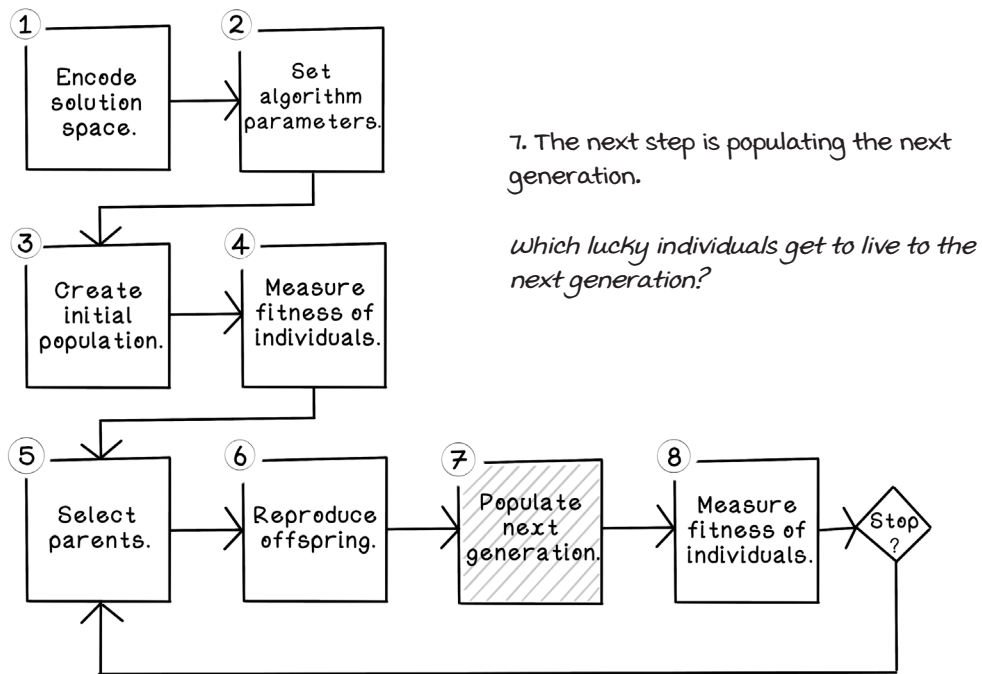


Figure 4.26 Populate the next generation.

The selection strategies mentioned in this section can be used to determine the individuals that are selected to form part of the population for the next generation.

Exploration vs. exploitation

Running a genetic algorithm always involves striking a balance between exploration and exploitation. The ideal situation is one in which there is diversity in individuals and the population as a whole seeks out wildly different potential solutions in the search space; then stronger local solution spaces are exploited to find the most desirable solution. The beauty of this situation is that the algorithm explores as much of the search space as possible while exploiting strong solutions as individuals evolve (figure 4.27).

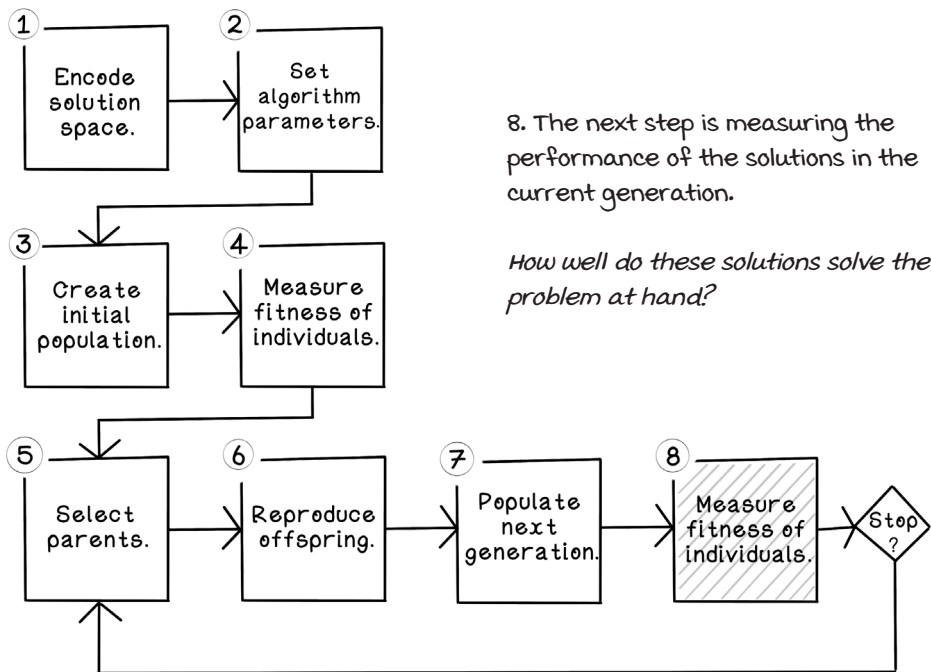


Figure 4.27 Measure the fitness of individuals.

Stopping conditions

Because a genetic algorithm is iterative in finding better solutions through each generation, a stopping condition needs to be established; otherwise, the algorithm might run forever. A *stopping condition* is the condition that is met where the algorithm ends; the strongest individual of the population at that generation is selected as the best solution.

The simplest stopping condition is a *constant*—a constant value that indicates the number of generations for which the algorithm will run. Another approach is to stop when a certain fitness is achieved. This method is useful when a desired minimum fitness is known but the solution is unknown.

Stagnation is a problem in evolutionary algorithms in which the population yields solutions of similar strength for several generations. If a population stagnates, the likelihood of generating strong solutions in future generations is low. A stopping condition could look at the change in the fitness of the best individual in each generation and, if the fitness changes only marginally, choose to stop the algorithm.

Pseudocode

The various steps of a genetic algorithm are used in a main function that outlines the life cycle in its entirety. The variable parameters include the population size, the number of generations for the algorithm to run, and the knapsack capacity for the fitness function, in addition to the variable crossover position and mutation rate for the crossover and mutation steps:

```
run_ga (population_size, number_of_generations, knapsack_capacity):
    let best_global_fitness equal 0
    let global_population equal...
    ...generate_initial_population(population_size)
    for generation in range(number_of_generations):
        let current_best_fitness equal...
        ...calculate_population_fitness(global_population, knapsack_capacity)
        if current_best_fitness is greater than best_global_fitness:
            let best_global_fitness equal current_best_fitness
        let the_chosen equal...
        ...roulette_wheel_selection(global_population, population_size)
        let the_children equal...
        ...reproduce_children(the_chosen)
        let the_children equal...
        ...mutate_children(the_children)
        let global_population equal...
        ...merge_population_and_children(global_population, the_children)
```

As mentioned at the beginning of this chapter, the Knapsack Problem could be solved using a brute-force approach, which requires more than 60 million combinations to be generated and analyzed. When comparing genetic algorithms that aim to solve the same problem, we can see far more efficiency in computation if the parameters for exploration and exploitation are configured correctly. Remember, in some cases, a genetic algorithm produces a “good enough” solution that is not necessarily the best possible solution but is desirable. Again, using a genetic algorithm for a problem depends on the context (figure 4.28).

	Brute force	Genetic algorithm
Iterations	$2^{26} = 67,108,864$	10,000 - 100,000
Accuracy	100%	100%
Compute time	~7 minutes	~3 seconds
Best value	13,692,887	13,692,887

Figure 4.28 Brute-force performance vs. genetic algorithm performance

Configuring the parameters of a genetic algorithm

In designing and configuring a genetic algorithm, several decisions need to be made that influence the performance of the algorithm. The performance concerns fall into two areas: the algorithm should strive to perform well in finding good solutions to the problem, and the algorithm should perform efficiently from a computation perspective. It would be pointless to design a genetic algorithm to solve a problem if the solution will be more computationally expensive than other traditional techniques. The approach used in encoding, the fitness function used, and the other algorithmic parameters influence both types of performances in achieving a good solution and computation. Here are some parameters to consider:

- *Chromosome encoding*—The chromosome encoding method requires thought to ensure that it is applicable to the problem and that the potential solutions strive for global maxima. The encoding scheme is at the heart of the success of the algorithm.
- *Population size*—The population size is configurable. A larger population encourages more diversity in possible solutions. Larger populations, however, require more computation at each generation. Sometimes, a larger population balances out the need for mutation, which results in diversity at the start but no diversity during generations. A valid approach is to start with a smaller population and grow it based on performance.
- *Population initialization*—Although the individuals in a population are initialized randomly, ensuring that the solutions are valid is important for optimizing the computation of the genetic algorithm and initializing individuals with the right constraints.
- *Number of offspring*—The number of offspring created in each generation can be configured. Given that after reproduction, part of the population is killed off to

ensure that the population size is fixed, more offspring means more diversity, but there is a risk that good solutions will be killed off to accommodate those offspring. If the population is dynamic, the population size may change after every generation, but this approach requires more parameters to configure and control.

- *Parent selection method*—The selection method used to choose parents can be configured. The selection method must be based on the problem and the desired explorability versus exploitability.
- *Crossover method*—The crossover method is associated with the encoding method used but can be configured to encourage or discourage diversity in the population. The offspring individuals must still yield a valid solution.
- *Mutation rate*—The mutation rate is another configurable parameter that induces more diversity in offspring and potential solutions. A higher mutation rate means more diversity, but too much diversity may deteriorate good-performing individuals. The mutation rate can change over time to create more diversity in earlier generations and less in later generations. This result can be described as exploration at the start followed by exploitation.
- *Mutation method*—The mutation method is similar to the crossover method in that it is dependent on the encoding method used. An important attribute of the mutation method is that it must still yield a valid solution after the modification or assigned a terrible fitness score.
- *Generation selection methods*—Much like the selection method used to choose parents, a generation selection method must choose the individuals that will survive the generation. Depending on the selection method used, the algorithm may converge too quickly and stagnate or explore too long.
- *Stopping condition*—The stopping condition for the algorithm must make sense based on the problem and desired outcome. Computational complexity and time are the main concerns for the stopping condition.

Use cases for evolutionary algorithms

Evolutionary algorithms have a wide variety of uses. Some algorithms address isolated problems; others combine evolutionary algorithms with other techniques to create novel approaches to solving difficult problems, such as the following:

- *Predicting investor behavior in the stock market*—Consumers who invest make decisions every day about whether to buy more of a specific stock, hold on to what they have, or sell stock. Sequences of these actions can be evolved and

mapped to outcomes of an investor's portfolio. Financial institutions can use this insight to proactively provide valuable customer service and guidance.

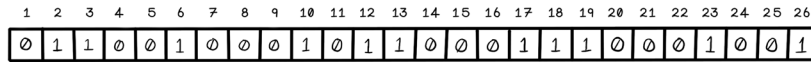
- *Feature selection in machine learning*—Machine learning is discussed in chapter 8, but a key aspect of machine learning is: given a number of features about something, determining what it is classified as. If we're looking at houses, we may find many attributes related to houses, such as age, building material, size, color, and location. But to predict market value, perhaps only age, size, and location matter. A genetic algorithm can uncover the isolated features that matter the most.
- *Code breaking and ciphers*—A *cipher* is a message encoded in a certain way to look like something else and is often used to hide information. If the receiver does not know how to decipher the message, it cannot be understood. Evolutionary algorithms can generate many possibilities for changing the ciphered message to uncover the original message.

Chapter 5 dives into advanced concepts of genetic algorithms that adapt them to different problem spaces. We explore different techniques for encoding, crossover, mutation, and selection, as well as uncover effective alternatives.

SUMMARY OF EVOLUTIONARY ALGORITHMS

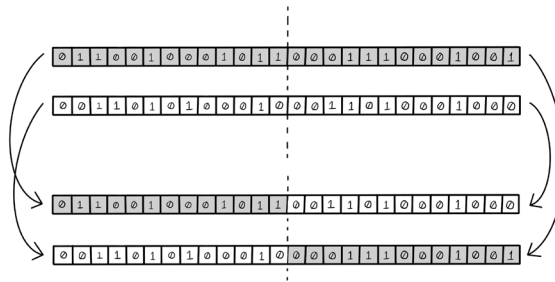
GAs use smart randomness to find good solutions fast.

Encoding is critical to the algorithm.

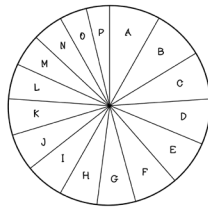


The fitness function is paramount to finding good solutions to the problem at hand.

Crossover aims to reproduce better solutions at each generation.



Selection favors stronger individuals but gives weaker ones an opportunity to possibly reproduce good solutions in the future.



Roulette-wheel selection

Explore at the start and exploit toward the end.

